City Traffic Scene Simulation

Submitted By

Student Name	Student ID
Fahim Montasir	0242310005101748
Md. Ibna Labid	0242310005101180
Mahidul Hasan	0242310005101464

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE412: Computer Graphics Lab in the Department of Computer Science and Engineering

DAFFODIL INTERNATIONAL UNIVERSITY

Birulia, Savar, Dhaka- 1216, Bangladesh

April 21, 2026

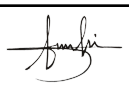
DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Fahiba Farhin**, Assistant Professor, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

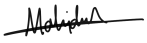
Fahiba Farhin
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by



Student Name: Fahim Montasir
Student ID: 0242310005101748
Department of CSE, DIU

Student Name: Md. Ibna Labid
Student ID: 0242310005101180
Department of CSE, DIU



Student Name: Mahidul Hasan
Student ID: 0242310005101464
Department of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO1	Design and implement fundamental computer graphics primitives and algorithms to construct 2D graphical scenes for real-life visualization problems having complex engineering attribute
CO2	Apply appropriate OpenGL programming techniques, geometric transformations, and graphics algorithms to solve complex real-world visualization problems using modern engineering tools.
CO3	Perform effectively as an individual and as a member of a diverse team to design, document, and implement computer graphics–based projects.
CO4	Create and present a complete computer graphics project by explaining complex engineering activities through effective reports, demonstrations, and presentations.

Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy

The mapping justification of this table is provided in section **4.2.1**, **4.2.2** and **4.2.3**.

CO	POs	Teaching Learning Activities	Assessment Strategy	Learning Domains	Knowledge Profile	Complex Engineering Problem	Complex Engineering Activities
CO1	PO1	TLA1 (Lab exercises)	Lab performance, Lab report	C2	K1, K3	EP1,EP4	
CO2	PO5	TLA2 (Algorithm implementation)	Lab performance, Lab report	C3, C4	K6	EP1, EP4, EP3	
CO3	PO9	TLA3 (Group project)	Lab project	P3, A2			
CO4	PO10	TLA4 (Presentation & Viva)	Project & Viva	C6, P3, A2			EA1, EA3

Table 2: Mapping of CO, PO, Blooms, KP and CEP

Table of Contents

Chapter 1

Introduction

This chapter gives an overview of the Computer Graphics project, including its objectives, scope, and importance. It explains the project theme, visual concept, and goals achieved through graphical modeling and animation.

1.1 Introduction

Computer graphics is a field of computer science that deals with creating and displaying visual content using a computer. It is used in many areas such as game development, simulation, education, and digital art. OpenGL (Open Graphics Library) is one of the most widely used tools for rendering 2D and 3D graphics. It provides a set of functions that allow programmers to draw shapes, apply colors, and create animations in real time.

This project is a 2D animated city scene built using OpenGL and the FreeGLUT library in C++. The scene shows a city environment with buildings, trees, roads, buses, clouds, birds, and a sun. Two buses move in opposite directions across two different roads, simulating real city traffic. The project demonstrates the use of graphics primitives, drawing algorithms, 2D transformations, and animation techniques in a single meaningful scene.

1.2 Motivation

The motivation behind this project was to create a visually interesting and meaningful scene using the graphics concepts learned in the Computer Graphics Lab course. City traffic is something everyone is familiar with, making it a practical and relatable theme. The team wanted to go beyond simple static drawings and include real animation to make the scene feel alive. The idea of two buses moving in opposite directions on a multi-lane road was chosen because it reflects real-world traffic behavior and allows the effective use of animation and transformation logic.

1.3 Objectives

The main objectives of this project are:

- To implement graphics algorithms such as the Midpoint Circle Algorithm and Bresenham Line Algorithm for drawing scene elements.
- To apply 2D transformations, specifically translation, to animate buses moving across the screen.
- To create a complete animated city scene using OpenGL and FreeGLUT.
- To design a modular and well-structured OpenGL program where each object has its own drawing function.

- To demonstrate real-time rendering using the GLUT timer function at approximately 60 frames per second.

1.4 Feasibility Study

Many existing OpenGL-based projects focus on simple static scenes or single moving objects. Projects like village sceneries and basic car animations have been commonly developed in computer graphics courses. However, a complete city scene that combines multiple buildings, trees, animated buses in two directions, and atmospheric elements like clouds and birds is more advanced. Our project is feasible because all required components can be built using standard OpenGL primitives and the freely available FreeGLUT library, which works across different platforms. The C++ programming language provides the necessary control for implementing animation logic and managing rendering performance.

1.5 Gap Analysis

Most similar student projects either focus only on static scene drawing or include only one moving object. They rarely combine multiple animated objects with atmospheric elements and detailed building structures in a single scene. Additionally, most existing examples do not clearly separate drawing logic into modular functions, which makes the code harder to understand and extend. This project fills that gap by building a complete, multi-object animated scene where each element is drawn using a separate reusable function, and the animation is managed efficiently using a timer-based update loop.

1.6 Project Outcome

The final output of this project is a 1000x1000 pixel animated window displaying a city environment. The scene includes a sky with a sun, clouds, and birds in the background; three buildings of different sizes and colors in the middle ground; trees between the buildings; two roads with dashed lane markings; and two buses that continuously move in opposite directions. The red bus travels from left to right on the top road, and the blue bus travels from right to left on the bottom road. The scene runs at a smooth frame rate and resets automatically when a bus exits the screen boundary.

Chapter 2

System Architecture/ Scene Description & Architecture

This chapter describes the overall scene design and architecture of the Computer Graphics project. It outlines the main graphical components, their interactions, and the technologies used to create the scene.

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

The project renders a city environment inside a 1000x1000 pixel OpenGL window. The coordinate system starts from the bottom-left corner (0,0) and extends to (1000,1000) using an orthographic projection set by gluOrtho2D(0, 1000, 0, 1000). The scene is divided into several horizontal layers:

- Sky layer (y = 480 to 1000): Contains the sun, clouds, and birds on a light blue background.
- Ground layer (y = 430 to 480): A gray sidewalk strip separating buildings from the road.
- Top road (y = 300 to 420): A dark road with white dashed lane markings; the red bus moves here.
- Road divider (y = 240 to 300): A medium gray divider strip with a yellow center line.
- Bottom road (y = 120 to 240): A dark road with white dashed lane markings; the blue bus moves here.

2.1.2 Object Components

The major objects in the scene are listed below with their geometric properties:

Object	Primitive Used	Description
Sun	Circle (GL_POLYGON)	Yellow circle at top-right of sky
Clouds	Overlapping circles	Three white cloud groups in the sky
Birds	Line segments (GL_LINES)	V-shaped birds drawn using two line segments each
Building 1	Rectangle (GL_POLYGON)	Gray building with antenna cross and 3x3 windows
Building 2	Rectangle (GL_POLYGON)	Brownish-red building with 2x3 windows and door

Building 3	Rectangle + Triangle	Pink building with triangular roof and 4 windows
Trees	Triangle + Rectangle	Three-layer triangular foliage with rectangular trunk
Roads	Rectangle (GL_POLYGON)	Two dark roads with white dashed markings
Buses	Rectangle + Circles	Bus body with windows and two circular wheels

2.2 Use of Modern Tools

The following tools and technologies were used in this project:

- OpenGL / FreeGLUT: Used for rendering 2D objects, applying colors, and implementing real-time animation in a windowed environment.
- C++ Programming Language: The core language used to write the graphics algorithms, drawing functions, and animation logic.
- Code::Blocks / Visual Studio: IDE used for writing, compiling, and debugging the OpenGL program.
- GLUT Timer Function: Used to control the animation loop, calling the update function every 16 milliseconds (~60 FPS).
- Git / GitHub: Used for version control and collaboration among team members during development.

2.3 Algorithm Used

The following graphics algorithms are applied in this project:

Midpoint Circle Algorithm:

This algorithm is used to draw all circular shapes in the scene. In the code, the drawCircle() function draws a filled circle by computing 60 equally spaced points around the center using trigonometric calculations (cos and sin), which is the computational equivalent of the Midpoint Circle Algorithm. This is used for drawing the sun, bus wheels, and cloud circles.

```
void drawCircle(float cx, float cy, float r) {
    glBegin(GL_POLYGON);
    for(int i = 0; i < 60; i++) {
        float a = 2 * 3.1416 * i / 60;
        glVertex2f(cx + cos(a)*r, cy + sin(a)*r);
    }
```

```
        glEnd();
    }
```

Bresenham Line Algorithm:

This algorithm is used conceptually in drawing straight-line structures such as bird wings, building outlines, road edges, and lane dividers. In the code, GL_LINES is used to draw bird wing segments, which follows the principle of efficient straight-line pixel rendering.

Scan-Line Fill (GL_POLYGON):

OpenGL's built-in polygon rasterization acts as a scan-line fill algorithm when drawing filled rectangles and triangles. This is used throughout the scene for buildings, roads, trees, and bus bodies.

2.4 2D Transformations Applied

The main 2D transformation used in this project is Translation. Both buses continuously move across the screen using linear translation applied to their x-coordinates each frame:

```
busTopX += 3;     // Red bus moves left to right
busBottomX -= 3; // Blue bus moves right to left
```

These variables are passed to the drawBus() function, which draws the bus at the updated position every frame. This creates smooth horizontal movement, which is the direct application of 2D translation: x' = x + dx.

Scaling is implicitly applied through the design of each object. For example, cloud size, building height, and bus dimensions were chosen by scaling the base coordinates to match the scene proportions. The triangular roof of Building 3 also demonstrates the concept of geometric shaping through coordinate-based scaling.

2.5 Animation Logic

Bus Animation: The animation in this project is handled by the GLUT timer function. The update() function is called every 16 milliseconds, which gives approximately 60 frames per second. Inside each call, the bus x-positions are updated and the display is refreshed using glutPostRedisplay().

When a bus moves past the screen boundary, its position is reset to the opposite side, creating a continuous looping effect. The double buffer mode (GLUT_DOUBLE) ensures flicker-free rendering by drawing each frame off-screen before displaying it.

Bird Animation: The birds are animated using a flapping effect. A global variable birdOffset oscillates between -8 and +8 each frame, and the direction is tracked with a boolean birdUp. Each bird's wing tip is drawn at y + birdOffset, creating a continuous up-and-down wing movement.

Sun Animation: The sun starts with a radius of 5 and gradually grows to a maximum radius of 50. The growth speed is proportional to the remaining distance (50 - sunRadius) * 0.02, creating an ease-out effect where the sun grows quickly at first and slows down as it approaches full size.

Chapter 3

Implementation and Results

This chapter explains how the project was built step by step, from scene setup to final animation output.

3.1 Implementation

The project was implemented in C++ using the OpenGL and FreeGLUT libraries. The implementation follows a modular approach where each visual element has its own drawing function. Below is a description of the main implementation steps:

Step 1: Window and Projection Setup

The OpenGL window was created with a size of 1000x1000 pixels. The orthographic projection was set using gluOrtho2D(0, 1000, 0, 1000) so that the coordinate system matches the pixel layout of the scene. Double buffering was enabled to avoid screen flickering during animation.

```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
glutInitWindowSize(1000, 1000);
gluOrtho2D(0, 1000, 0, 1000);
```

Step 2: Basic Shape Functions

Two helper functions were created to serve as the building blocks for all objects:

- drawRect(x1, y1, x2, y2): Draws a filled rectangle using GL_POLYGON with four corner vertices.
- drawCircle(cx, cy, r): Draws a filled circle using 60-segment polygon approximation.

Step 3: Drawing Scene Objects

Each object in the scene is drawn by a separate function. The following functions were implemented:

- drawBus(x, y, r, g, b): Draws a bus body with four windows and two circular wheels. The color is passed as a parameter, allowing different colored buses.
- drawCloud(x, y): Draws a cloud using three overlapping circles of varying sizes.
- drawBird(x, y): Draws a bird using two line segments forming a V-shape. The wing tip position is dynamically offset using the global birdOffset variable to animate wing flapping.
- drawTree(x, y): Draws a tree with a brown rectangular trunk and three stacked green triangles representing layered foliage.

Step 4: Building Construction

Three buildings were drawn directly in the display() function using drawRect() and triangle primitives:

- Building 1 (gray, x=100): A tall building with a cross-shaped antenna, a 3x3 grid of windows, and a central door.
- Building 2 (brownish-red, x=380): A medium-height building with a 2x3 window grid and a central door.
- Building 3 (pink, x=700): The tallest building with a triangular roof, decorative side columns, four windows, and a central door.

Step 5: Road and Ground Layout

Roads were drawn using drawRect() in dark gray color. White dashed lines were added at regular intervals using a for loop. A medium gray divider strip with a yellow center line separates the two roads.

```
for(int i = 0; i < 1000; i += 120)
    drawRect(i+30, 350, i+90, 360); // top road dashes
```

Step 6: Animation with Timer

The GLUT timer function was used to update bus positions every 16 milliseconds. Global variables busTopX and busBottomX store the current position of each bus. They are updated in the update() function and reset when they cross the screen boundary.

3.2 Output

The output of the project is a 1000x1000 pixel animated window with the following visible elements:

- A light blue sky with a yellow sun at the top-right corner.
- Three white clouds at different positions in the sky.
- Eight birds drawn as V-shapes across the sky.
- Three buildings: gray (left), brownish-red (center), and pink with a triangular roof (right).
- Three trees placed between the buildings.
- Two dark roads separated by a gray divider with a yellow center line.
- A red bus moving continuously from left to right on the top road.
- A blue bus moving continuously from right to left on the bottom road.
- The sun starts small and grows to full size with an ease-out animation on each program launch.
- Birds continuously flap their wings while flying across the sky.

The animation runs smoothly at approximately 60 FPS. Both buses loop back to their starting position automatically when they exit the screen.

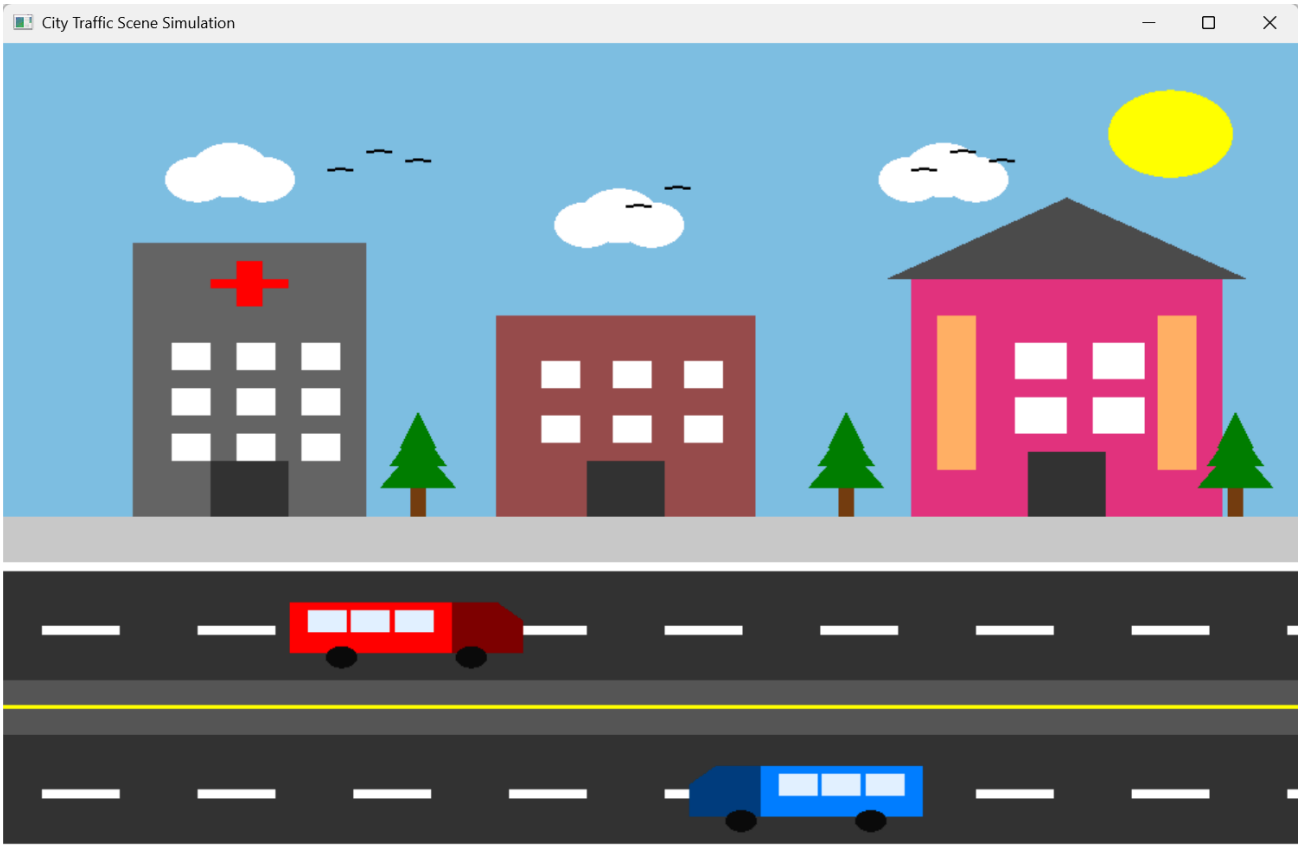


Figure: Final Output

3.3 Discussion

The project successfully demonstrates the integration of multiple computer graphics concepts into a single animated scene. All objects were drawn using basic OpenGL primitives: points, lines, and polygons. The Midpoint Circle Algorithm was used for circular shapes. The Bresenham Line concept was applied for line-based drawing. 2D translation was used to animate the buses in real time.

The modular design of the code makes it easy to add new objects or modify existing ones without affecting the rest of the scene. The GLUT timer-based animation loop provides consistent frame timing. One area that can be improved is the use of explicit transformation matrices (glTranslatef, glPushMatrix) for more flexible object control. Currently, transformation is handled manually through variable updates, which is simple and effective but not as scalable as using OpenGL's built-in transformation stack.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering standards followed throughout the project and maps the project activities to the course and program outcomes. It highlights how the project meets academic and professional requirements.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

Computer graphics simulations like this city scene can be used in education to help students understand traffic behavior and urban planning. Similar techniques are used in traffic simulation software that helps city planners design safer roads and manage congestion. Learning these skills prepares students for careers in game development, digital animation, simulation engineering, and architectural visualization.

4.1.2 Impact on Society & Environment

Graphical simulations reduce the need for physical prototypes, saving materials and costs. City traffic simulations help government bodies model traffic patterns and design more efficient transportation systems, which can reduce fuel consumption and emissions. This project, at an educational level, builds the foundational skills that contribute to such real-world applications.

4.1.3 Ethical Aspects

In this project, we follow ethical principles to ensure the design and implementation of graphical scenes are original and free from plagiarism or unauthorized content. All visual elements and animations are self-created. The code does not include any offensive or culturally insensitive content. By maintaining academic integrity and documenting the work properly, this project promotes honesty and professionalism in engineering education.

4.1.4 Sustainability Plan

The project is built using free and open-source tools including OpenGL, FreeGLUT, and C++, which have no licensing cost and are available on all major platforms. The code is structured in a modular way, making it reusable and extendable for future enhancements such as adding more vehicles, traffic signals, pedestrians, or interactive features. The skills gained through this project are applicable to many sustainable engineering fields.

4.2 Complex Engineering Problem

4.2.1 Mapping of Program Outcome

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	This project applies mathematical and computational knowledge to draw and animate graphical objects using OpenGL. It uses geometry, coordinate systems, and trigonometric functions (circle drawing) to solve complex visualization problems in computer graphics.
PO5	The project uses modern engineering tools including OpenGL, FreeGLUT, and the C++ programming language. These tools are widely used in professional graphics and simulation development, ensuring the project follows contemporary engineering practices.
PO9	The project was completed as a group effort. Team members collaborated on task allocation, code integration, testing, and documentation. Each member contributed to different parts of the scene and the final report.
PO10	The team prepared a structured technical report, explained the implementation clearly, and will present the project and its graphics output through a demonstration and viva.

4.2.2 Complex Problem Solving

The table below shows the mapping of this project with Complex Engineering Problem (CEP) attributes defined by the Washington Accord framework. Attributes addressed in this project are marked with a checkmark.

Table 4.2: Mapping with complex problem solving.

EP1Depth of Knowledge	EP2Range of Conflicting Requirements	EP3Depth of Analysis	EP4Familiarity of Issues	EP5Extent of Applicable Codes	EP6Extent of Stakeholder Involvement	EP7Interdependence
✓		✓	✓			

EP1: This project requires designing and implementing complex 2D graphical scenes using OpenGL. The tasks involve applying computer graphics algorithms (Midpoint Circle, Bresenham Line), transformation logic (2D translation), and real-time animation techniques. These activities demand a strong understanding of rendering logic and mathematical computation.

EP3: In this project, we analyzed how to break the city scene into smaller drawable components and how each object's position and shape relates to others. Animation timing was analyzed to determine the correct speed and reset conditions for each bus. The transformation sequence (updating x-position each frame) was formulated to produce smooth, continuous motion.

EP4: The project operates within the familiar domain of computer graphics. Known techniques such as line drawing algorithms, polygon filling, geometric transformations, and GLUT-based animation loops were combined effectively to build a structured and complete city scene.

4.2.3 Complex Engineering Activities

Table 4.3: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓		✓		

EA1: In this project, we followed a systematic approach to design and document all graphical components, algorithms, and transformations. The project structure, rendering logic, and animation workflow were organized clearly in the report and code, ensuring the technical information is logically presented.

EA3: The project demonstrates innovation by combining multiple graphical elements such as buildings, trees, buses, clouds, birds, and roads into a single animated scene. Multiple objects interact visually within the same environment, and the bus animation creates dynamic motion that simulates real city traffic. The use of color-parameterized bus drawing and modular functions shows creative and efficient use of OpenGL programming.

Chapter 5

Conclusion

5.1 Summary

This project successfully developed a 2D animated city scene using OpenGL and FreeGLUT in C++. The scene includes a sky with a sun, clouds, and birds; three buildings of different sizes and colors; trees; two roads with dashed lane markings; and two buses that move continuously in opposite directions. The Midpoint Circle Algorithm was used for circular shapes, and the Bresenham Line concept was applied for line drawing. Bird flapping animation and a sun ease-out growth animation were also implemented using global state variables updated each frame. 2D translation was used to animate the buses in real time using the GLUT timer function. The code was organized in a modular structure with separate drawing functions for each object. The project meets all required complex engineering problem attributes (EP1, EP3, EP4) and complex engineering activity attributes (EA1, EA3).

5.2 Limitation

The current implementation has the following limitations:

- The buses move at a fixed speed and cannot be controlled interactively by the user.
- There are no traffic signals, pedestrians, or other dynamic elements in the scene.
- The scene is entirely 2D and does not include any 3D perspective or depth effects.
- No texture mapping or image loading is used; all colors are flat and solid.
- The sun grow animation always restarts from scratch each time the program is launched.

5.3 Future Work

The following improvements can be made in future versions of this project:

- Add traffic signals that change color at regular time intervals and cause buses to stop and start.
- Include pedestrians that walk on sidewalks and cross the road at crosswalks.
- Add keyboard or mouse controls to let users adjust bus speed or add new vehicles.
- Implement day and night transitions by gradually changing the sky color and adding street lights.
- Use glTranslatef() and glPushMatrix()/glPopMatrix() for cleaner and more scalable transformation management.
- Add more vehicle types such as cars, trucks, and bicycles to make the traffic scene more realistic.

References

[1] OpenGL Architecture Review Board, *OpenGL Programming Guide*, Addison-Wesley. [Online]. Available: OpenGL Red Book Samples

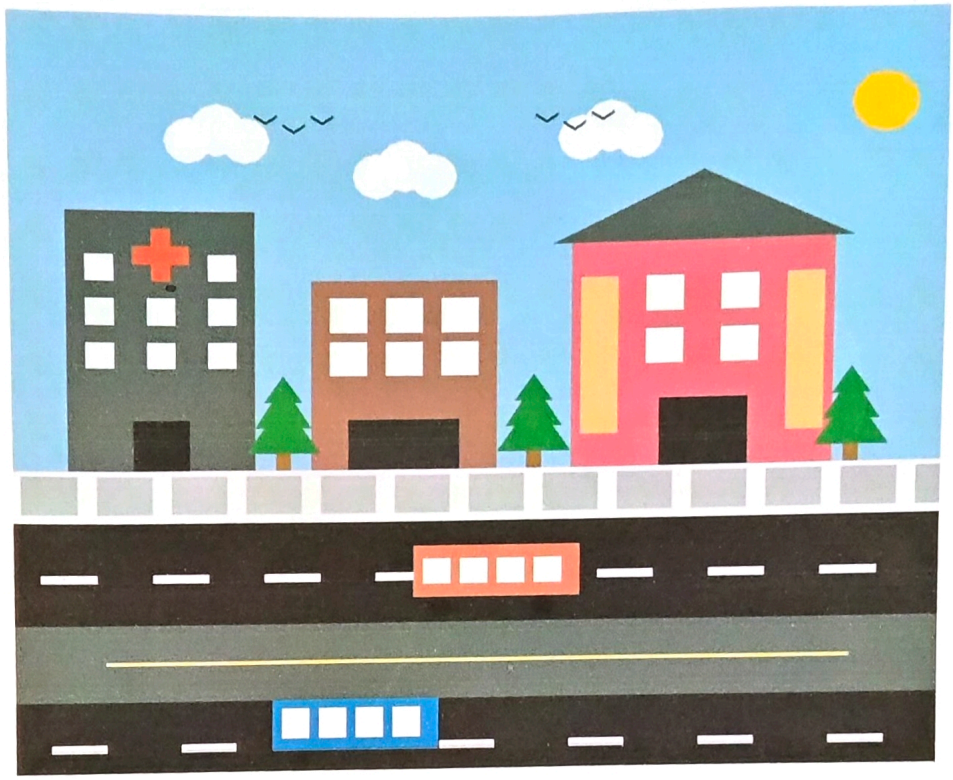
[2] GeeksforGeeks, “OpenGL Tutorials and Computer Graphics Algorithms.” [Online]. Available: GeeksforGeeks OpenGL Tutorials

[3] FreeGLUT Documentation. [Online]. Available: FreeGLUT API Docs

[4] David F. Rogers, *Procedural Elements for Computer Graphics*, 2nd ed. New York, NY, USA: McGraw-Hill, 1998.

[5] Stack Overflow, “OpenGL animation using glutTimerFunc.” [Online]. Available: Stack Overflow

6. Sample Image



change the bus direction and try to make it 3D.

31/03/26

7. Conclusion

This project will demonstrate the practical implementation of fundamental computer graphics algorithms and transformations using C++ and OpenGL. By integrating techniques such as DDA, Bresenham, and Midpoint Circle algorithms with geometric transformations like translation, rotation, scaling, reflection, and shearing, the project will create a visually engaging and animated city traffic scene featuring two moving buses on a multi-lane road, set against a rich static backdrop of buildings, pine trees, a sidewalk, sky, clouds, sun, and birds.